CPE 486/586: Deep Learning for Engineering Applications

10 Physics-informed Neural Networks

Spring 2026

Rahul Bhadani

Outline

Motivation

Solving a Differential Equation

Consider a differential equation:

$$\frac{dx(t)}{dt} = -2x$$

How do we solve this Equation?

Solving a Differential Equation

Consider a differential equation:

$$\frac{dx(t)}{dt} = -2x$$

How do we solve this Equation?

Analytically, it is $x = e^{-2t}$ with initial condition $x(0) = 1$.

Solving a Differential Equation

Consider a differential equation:

$$\frac{dx(t)}{dt} = -2x$$

How do we solve this Equation?

Analytically, it is $x = e^{-2t}$ with initial condition $x(0) = 1$.

But it is usually not analytically possible to solve all kind of differential equations.

Now Data Enters the Picture

Now, think of a situation when we know x from some measurement of the process and also assume that it follows the differential equation provided earlier.

Now Data Enters the Picture

Now, think of a situation when we know x from some measurement of the process and also assume that it follows the differential equation provided earlier.

Here, the independent variable is t, we know x and we want to find f such that it can give x for any value of t.

Now Data Enters the Picture

Now, think of a situation when we know x from some measurement of the process and also assume that it follows the differential equation provided earlier.

Here, the independent variable is t, we know x and we want to find f such that it can give x for any value of t.

What's the problem then?

Now Data Enters the Picture

Now, think of a situation when we know x from some measurement of the process and also assume that it follows the differential equation provided earlier.

Here, the independent variable is t, we know x and we want to find f such that it can give x for any value of t.

What's the problem then?

It may not necessarily be the solution to the differential equation provided earlier!

Differential Equation as Regularizer

Let's rewrite our differential equation

Consider a differential equation:

$$\frac{dx(t)}{dt} + 2x = 0$$

But as the real world data will exhibit some error, $x(t)$ might not satisfy the above equation. But if we approximate $x(t)$ using some neural network $u^{\theta}(x,t)$, we would want

$$\frac{du^{\theta}(x,t)}{dt} + 2u^{\theta}(x,t) \leq \varepsilon$$

where ε

Differential Equation as Regularizer

Hence, apart from the training loss, we introduce a physics loss

Consider a differential equation:

$$\mathcal{L}_{physics}$$

which is

$$\mathcal{L}_{physics} = \frac{du^{\theta}(x,t)}{dt} + 2u^{\theta}(x,t)$$

for the previous example that we would be looking to minimize.

This is what we call **Physics-Informed Deep Learning (PIDL)** or in general **Physics Informed Machine Learning (PIML)** and the Neural Network used would be called **Physics-Informed Neural Network (PINN)**.

Updated Loss Function

$$\mathcal{L} = \lambda_1 \mathcal{L}_{data} + \lambda_2 \mathcal{L}_{physics} + \lambda_3 \mathcal{L}_{IC} + \lambda_4 \mathcal{L}_{BC}$$

where IC is for initial conditions and BC is for boundary conditions.

Data + Physics

Physics governs almost every system we want to model.

Classical numerical solvers (FEM, FDM, FVM):

- Require fine meshes
- Scale poorly to high dimensions
- Cannot easily solve inverse problems
- Need full boundary conditions

Pure data-driven ML:

- Needs enormous labelled data
- No physics guarantee
- Fails to extrapolate
- Black-box predictions

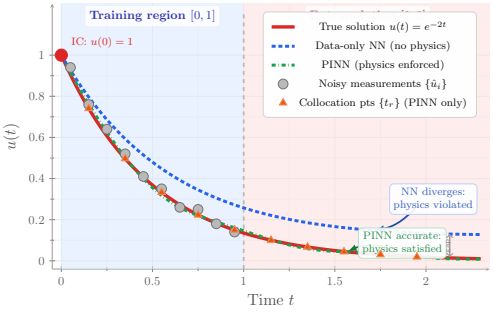
Can we get the best of both worlds?

Data Alone is Not Enough

Suppose we fit a neural network $u^\theta(x,t)$ purely on noisy measurements $\{(t_i, \hat{x}_i)\}$.

Problems:

- The network may **overfit** to noise
- **Extrapolation** outside training region fails
- The prediction is **not guaranteed** to satisfy $\frac{dx}{dt} = -2x$
- Different networks with the same data error can give **physically inconsistent** results



Red: truth *Blue*: data-only NN *Green*: PINN

The Complete PINN Loss

We want $u^\theta(x,t)$ to simultaneously:

1. **Fit the data** at measurement points $\{t_i, \hat{x}_i\}$
2. **Satisfy the ODE** at collocation points $\{t_r\}$
3. **Satisfy the initial condition** $u^\theta(x,0) = x_0$

$$\mathcal{L}_{\text{total}}(\theta) = \underbrace{\lambda_d \frac{1}{N_d}\sum_{i=1}^{N_d}\left|u^\theta(x,t_i) - \hat{x}_i\right|^2}_{\mathcal{L}_{\text{data}}} + \underbrace{\lambda_r \frac{1}{N_r}\sum_{j=1}^{N_r}\left|\frac{du^\theta(x,t_j)}{dt}\Big|_{t_j} + 2u^\theta(x,t_j)\right|^2}_{\mathcal{L}_{\text{physics}}} + \lambda_0 \underbrace{\left|u^\theta(x,0) - 1\right|^2}_{\mathcal{L}_{\text{IC}}}$$

The Complete PINN Loss

We want $u^\theta(x,t)$ to simultaneously:

1. **Fit the data** at measurement points $\{t_i, \hat{x}_i\}$
2. **Satisfy the ODE** at collocation points $\{t_r\}$
3. **Satisfy the initial condition** $u^\theta(x,0) = x_0$

$$\mathcal{L}_{\text{total}}(\theta) = \underbrace{\lambda_d \frac{1}{N_d}\sum_{i=1}^{N_d}\left|u^\theta(x,t_i) - \hat{x}_i\right|^2}_{\mathcal{L}_{\text{data}}} + \underbrace{\lambda_r \frac{1}{N_r}\sum_{j=1}^{N_r}\left|\frac{du^\theta(x,t_j)}{dt}\Big|_{t_j} + 2u^\theta(x,t_j)\right|^2}_{\mathcal{L}_{\text{physics}}} + \lambda_0 \underbrace{\left|u^\theta(x,0) - 1\right|^2}_{\mathcal{L}_{\text{IC}}}$$

The physics loss needs **derivatives of the network output** — these are computed *exactly* via **automatic differentiation**.

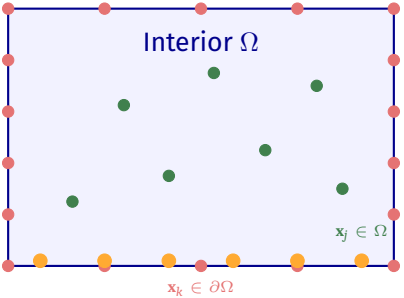
The Complete PINN Loss: General Case

$$\mathcal{L}_{\text{total}}(\theta) = \underbrace{\frac{\lambda_d}{N_d}\sum_{i=1}^{N_d}\left|u^\theta(\mathbf{x}_i,t_i)-\hat{u}_i\right|^2}_{\mathcal{L}_{\text{data}}} + \underbrace{\frac{\lambda_r}{N_r}\sum_{j=1}^{N_r}\left|\Psi\left[u^\theta\right](\mathbf{x}_j,t_j)-f(\mathbf{x}_j,t_j)\right|^2}_{\mathcal{L}_{\text{physics}}}$$

$$+ \underbrace{\frac{\lambda_b}{N_b}\sum_{k=1}^{N_b}\left|\mathcal{B}\left[u^\theta\right](\mathbf{x}_k,t_k)-g(\mathbf{x}_k,t_k)\right|^2}_{\mathcal{L}_{\text{BC}}} + \underbrace{\frac{\lambda_0}{N_0}\sum_{m=1}^{N_0}\left|u^\theta(\mathbf{x}_m,0)-u_0(\mathbf{x}_m)\right|^2}_{\mathcal{L}_{\text{IC}}}$$

Symbol	Meaning	Evaluated on
$\Psi[\cdot]$	Differential operator (PDE/ODE)	Interior $\Omega \times (0,T]$
$\mathcal{B}[\cdot]$	Boundary operator (Dirichlet / Neumann / Robin)	$\partial\Omega \times (0,T]$
$f(\mathbf{x},t)$	Forcing / source term	
$g(\mathbf{x},t)$	Prescribed boundary value	
$u_0(\mathbf{x})$	Initial condition	$\Omega \times \{t=0\}$
$\lambda_d, \lambda_r, \lambda_b, \lambda_0$	Loss weights (hyperparameters)	

Sampling Sets for Each Loss Term



- Collocation (physics)
- Boundary (BC)
- Initial time $t = 0$ (IC)
- Measurement (data, if available)

None of these sets require a mesh — points can be placed **anywhere**, including adaptively where the residual is largest.

Two Modes: Forward and Inverse

Forward Problem

Given: The PDE + BCs/ICs
Find: The solution $u(x, t)$

Physics $\longrightarrow$ Solution

Inverse Problem

Given: Sparse measurements of u
Find: Unknown PDE parameters γ

Measurements $\longrightarrow$ Physics

Two Modes: Forward and Inverse

Forward Problem

Given: The PDE + BCs/ICs
Find: The solution $u(x, t)$

Physics $\longrightarrow$ Solution

Inverse Problem

Given: Sparse measurements of u
Find: Unknown PDE parameters γ

Measurements $\longrightarrow$ Physics

PINNs handle both in the same framework
(just add unknown parameters λ as trainable variables)

0x00FF00FF

We want data-efficient universal function approximators that naturally encode any underlying physical laws as prior information.

0xABCD1234

Differentiable Physics

We use domain knowledge in the form of model equations.

- We incorporate discretized version of this model equations into our training process.

In a supervised settings, we are interested in

$$f : X \rightarrow Y$$

Let's assume Φ be a generic differential equations such that

$$\Phi : Y \rightarrow Z$$

which encodes a property of the solutions, e.g. some real world behavior we'd like to match.

> In contrast to methods discussed in previous chapters, we use discretized version of Φ, and and employ it to guide the training of f.

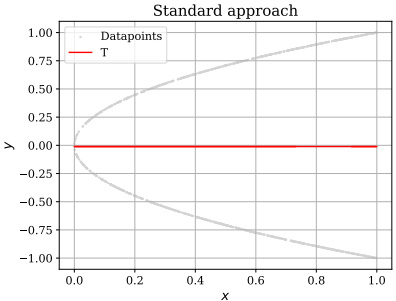
Example 1: Finding The Inverse Function of Parabola

Given $\Phi : x \rightarrow x^2$.
We need to find a function $f \approx \Phi^{-1}$ such that $\Phi(f(x)) = x$ (that is $f(x) \approx \pm\sqrt{x}$. A neural network f_θ is trained to approximate this.

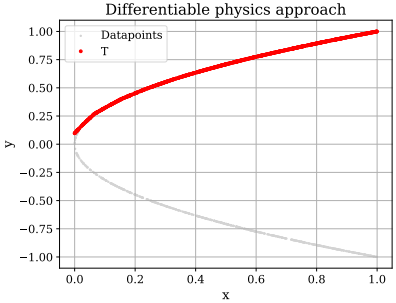
Example 1: Finding The Inverse Function of Parabola

Given $\Phi : x \to x^2$.
We need to find a function $f \approx \Phi^{-1}$ such that $\Phi(f(x)) = x$ (that is $f(x) \approx \pm\sqrt{x}$. A neural network f_θ is trained to approximate this.

Data is $\mathcal{D}\{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$ where $y^{(i)} = \pm\sqrt{x}^{(i)}$.



Loss: $\frac{1}{n}\sum_{i=1}^{N}(f_\theta(x^{(i)}) - y^{(i)})^2$



Loss: $\frac{1}{n}\sum_{i=1}^{N}(\Phi(f_\theta(x^{(i)})) - x^{(i)})^2$.

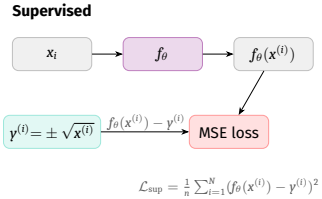
Here, no $y^{(i)}$ is used, but only the input $x^{(i)}$ and known forward operator Φ.

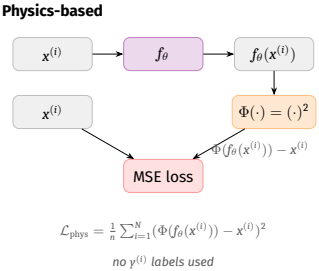
Backpropagation in Physics Loss

For the physics loss, backprop through Φ gives

$$\frac{\partial \mathcal{L}_{phys}}{\partial f_\theta(x^{(i)})} = 4 f_\theta(x^{(i)})\left(f_\theta(x^{(i)})^2 - x^{(i)}\right)$$

Its derivative vanishes at $f_\theta(x^{(i)}) = 0$, regardless of $x^{(i)}$. Hence, the zero function is a saddle point of the physics loss that doesn't affect the supervised loss.





Clearly, it didn't handle the **bifurcation**.

Can we do better?

Clearly, it didn't handle the **bifurcation**.

Can we do better?

We can learn the full distribution of the posterior.

Clearly, it didn't handle the **bifurcation**.

Can we do better?

We can learn the full distribution of the posterior.

Do you remember **Flow Matching**?

Flow Matching for Inverse Function of Parabola

Assuming, normalized time, i.e. $t \sim \mathcal{U}[0, 1]$.

Recall, data is $\mathcal{D}\{\mathbf{x}^{(i)} \equiv (x^{(i)}, y^{(i)})\}_{i=1}^{N}$ where $y^{(i)} = \pm\sqrt{x^{(i)}}$.

We define $\mathbf{x} \sim \mathcal{N}(0, \mathbf{I}_2)$. Initial condition $\mathbf{x}_0^{(i)}[0] = x^{(i)}$ (we fix x-coordinate noise).

For every sample index i, the end point should be $\mathbf{x}_1^{(i)} = (x^{(i)}, y^{(i)})^\top$.

As we need to define a probability path p_t, but as we understand the p_t being marginal is difficult to compute.

Conditional interpolant can help in this case. We fix a specific pair and define a deterministic path between them, such as,

$$\mathbf{x}_t\big|_{(\mathbf{x}_0, \mathbf{x}_1)} = (1 - t)\mathbf{x}_0 + t\mathbf{x}_1$$

But remember that the interpolant doesn't have to be linear.

Flow Matching for Inverse Function of Parabola

The marginal interpolant $p_t(\mathbf{x})$ is recovered by averaging over all pairs:

$$p_t(\mathbf{x}) = \int p_t(\mathbf{x}_t|_{(\mathbf{x}_0,\mathbf{x}_1)}) q(\mathbf{x}_0) p_{data}(\mathbf{x}_1) d\mathbf{x}_0 d\mathbf{x}_1$$

And, we have target conditional vector field as $u_t = \frac{dx_t}{dt}$, which is always deterministic, that is $x_1 - x_0$ (only applicable for linear interpolant, for non-linear interpolant, you will see t in RHS of u_t).

We can define a neural network $v_\theta(\mathbf{x}_t, t)$ that will regress u_t with Flow Matching Loss function:

$$\mathcal{L}_{FM}(\theta) = \frac{1}{n} \sum_{i=1}^{N} ||v_\theta(\mathbf{x}_t^{(i)}, t^{(i)}) - \mathbf{u}_t^{(i)}||^2$$

Flow Matching for Inverse Function of Parabola

At the inference time (prediction with the trained model), given $\mathbf{x}_0 = (x^{(i)}, z)^\top$ with $z \sim \mathcal{N}(0, 1)$, the integrate the learned ODE model with step size $\Delta t = \frac{1}{T}$:

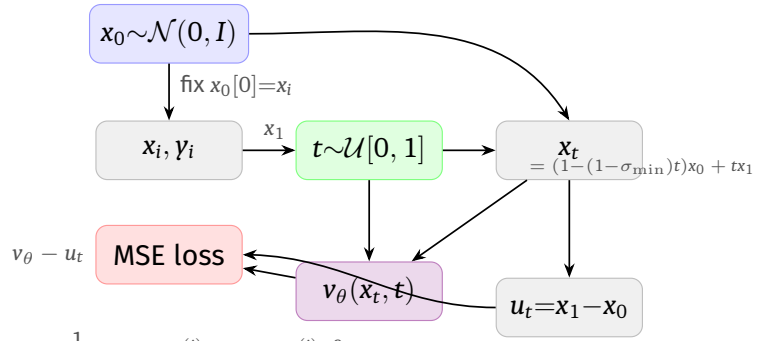
$$\mathbf{x}^{(i)}_{t_{k+1}} = \mathbf{x}^{(i)}_{t_k} + \Delta t v_\theta(\mathbf{x}^{(i)}_{t_k}, t_k)$$

and then reimpose the x-conditioning at every step: $\mathbf{x}^{(i)}_{t_{k+1}}[0] \leftarrow x^{(i)}$ and we get the second coordinate of the final state

$$\mathbf{x}^{(i)}_{t_1}[1] \sim p(y|x^{(i)})$$

Flow Matching for Inverse Function of Parabola

Flow Matching



$$\mathcal{L}_{\text{FM}} = \frac{1}{n} \sum_i \| v_\theta(x_t^{(i)}, t^{(i)}) - u_t^{(i)} \|^2$$

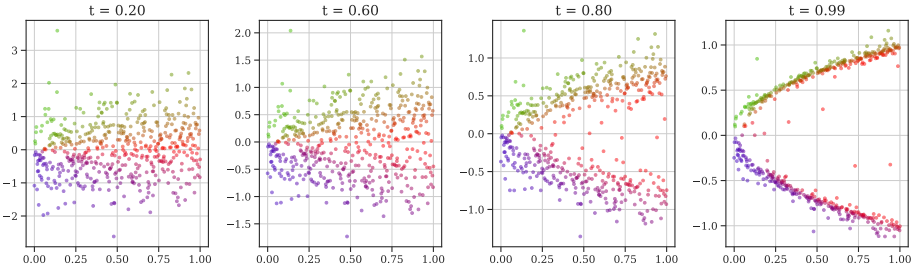
no y_i labels; t, x_0 stochastic

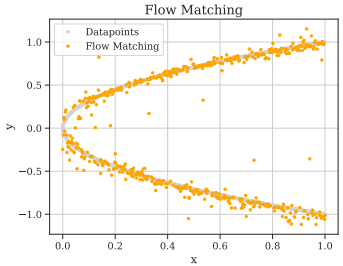
Flow Matching for Inverse Function of Parabola

Caveats about Flow Matching:

- Flow Matching is **not** PINN.
- Physics get introduced in data construction $y^{(i)} = \pm\sqrt{x^{(i)}}$ but FM never sees Φ.
- It learns conditional distributipn $p(y|x)$.
- In the case of FM, the parabola $y^2 = x$ is the **support manifold** of that distribution, but no operator passed to the loss.

Flow Matching for Inverse Function of Parabola





Clearly, Flow Matching provides an approximate solution, although allows to take into account Bifurcation but makes mistakes as well.

FM is not PINN.

References

- Physics-informed machine learning https://doi.org/10.1038/s42254-021-00314-5
- Physics Informed Deep Learning (Part I): Data-driven Solutions of Nonlinear Partial Differential Equations https://arxiv.org/abs/1711.10561
- Physics Informed Deep Learning (Part II): Data-driven Discovery of Nonlinear Partial Differential Equations https://arxiv.org/abs/1711.10566
- Thuerey, Nils, Philipp Holl, Maximilian Mueller, Patrick Schnell, Felix Trost, and Kiwon Um. "Physics-based deep learning." arXiv preprint arXiv:2109.05237 (2021). https://arxiv.org/abs/2109.05237
- Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations Author links open overlay panel https://doi.org/10.1016/j.jcp.2018.10.045
- https://github.com/maziarraissi/PINNs
- https://github.com/prateekbhustali/Physics-Informed-Neural-Networks
- https://github.com/nguyenkhoa0209/pinns_tutorial
- https://annien094.github.io/PINNs-tutorial-MICCAI-2024/
- Physics-informed neural networks for PDE problems: a comprehensive review https://doi.org/10.1023/a:1012784129883
- A hands-on introduction to Physics-Informed Neural Networks for solving partial differential equations with benchmark tests taken from astrophysics and plasma physics https://arxiv.org/pdf/2403.00599